WEEK 3 · LESSON 14 OF 24 · HANDOUT

Where Bias Comes From and How to Reduce It

Study time: **about 60 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=vq8mDiDODhc — open the lesson online to watch inside the course.

🎓 **Go deeper:** PyBay 2019: DebiasML — https://www.youtube.com/watch?v=VCyk0_I-8BM

🎓 **Go deeper:** PyCon 2019: Measures and Mismeasures of Algorithmic Fairness — https://www.youtube.com/watch?v=olCKhigTVuY

Your goal in this lesson: Show how historical bias and proxy features enter a model, apply three mitigation methods, and compare their effects on fairness and accuracy.

Sources of bias

Source	Example in lending
Historical bias	Training on past officers' decisions, which were unfair
Proxy features	Mobile-money activity is much lower in rural areas, so it "stands in" for area
Representation bias	Few applicants from one district, so the model learns little about them
Measurement bias	"Late payments" recorded more strictly in some branches
Feedback loops	Refused people never get loans, so they never build a repayment record

```
import numpy as np
import pandas as pd

def make_applicants(n=3000, seed=7, drift=0.0, concept=0.0):
    # Made-up SACCO loan applications. drift > 0: incomes, loans and late payments change.
    # concept > 0: the link to repayment changes (salaried jobs protect less, late payments
hurt more).
    rng = np.random.default_rng(seed)
    gender = rng.choice(["female", "male"], n)
    area = rng.choice(["urban", "rural"], n, p=[0.6, 0.4])
    rural = area == "rural"
    income = (rng.lognormal(12.4, 0.5, n) * np.where(rural, 0.7, 1.0) * (1 - drift *
0.25)).round(-3)
    loan = (rng.uniform(50_000, 600_000, n) * (1 + drift)).round(-3)
    years = rng.integers(0, 13, n)
    late = rng.poisson(0.7 + drift, n)
    salaried = (rng.random(n) < np.where(rural, 0.3, 0.6)).astype(int)
    momo = np.clip(rng.normal(np.where(rural, 18, 45), 10), 0, None).round().astype(int)
    score = 3.0 - 1.6 * (loan / income) + 0.12 * years - 0.7 * late + 0.6 * salaried + 0.01 *
momo
    score = score - concept * (1.5 * salaried + 0.6 * late - 0.04 * (momo - 30))
    repaid = (rng.random(n) < 1 / (1 + np.exp(-score))).astype(int)
    officer = score - 0.9 * rural - 0.6 * (gender == "female") + rng.normal(0, 0.6, n)
    return pd.DataFrame({
        "applicant_id": np.arange(1, n + 1), "age": rng.integers(20, 63, n), "gender": gender,
"area": area,
        "income_rwf": income, "loan_rwf": loan, "years_member": years, "late_payments": late,
        "salaried": salaried, "momo_tx_month": momo,
        "approved_before": (officer > 1.2).astype(int), "repaid": repaid})

applicants = make_applicants()
print(applicants.shape, "| repaid:", round(applicants["repaid"].mean(), 3))
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, roc_auc_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

FEATURES = ["income_rwf", "loan_rwf", "years_member", "late_payments", "salaried",
"momo_tx_month"]
TARGET = "repaid"
train, test = train_test_split(applicants, test_size=0.25, random_state=0,
stratify=applicants[TARGET])
model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))
model.fit(train[FEATURES], train[TARGET])
proba = model.predict_proba(test[FEATURES])[:, 1]
print(f"accuracy {accuracy_score(test[TARGET], proba >= 0.5):.3f} | AUC
{roc_auc_score(test[TARGET], proba):.3f}")
```

```
def group_report(data, group, decision, outcome="repaid"):
    rows = []
    for name, g in data.groupby(group):
        rows.append({
            group: name, "people": len(g),
            "approval_rate": g[decision].mean(),                              # selection rate
            "approved_if_would_repay": g.loc[g[outcome] == 1, decision].mean(), # true positive
rate
            "approved_if_would_default": g.loc[g[outcome] == 0, decision].mean(), # false
positive rate
            "repay_rate": g[outcome].mean(),
        })
    return pd.DataFrame(rows).set_index(group).round(3)

def fairness_summary(report):
    a, t = report["approval_rate"], report["approved_if_would_repay"]
    return {"approval_gap": round(float(a.max() - a.min()), 3),
            "disparate_impact": round(float(a.min() / a.max()), 3),
            "equal_opportunity_gap": round(float(t.max() - t.min()), 3)}
```

Historical bias: copying unfair decisions

Many organisations train on **past decisions** because real outcomes are missing for refused people. Our data has `approved_before`: past loan-officer decisions that were secretly unfair to rural applicants and women.

```
past = applicants.groupby(["area", "gender"])[["approved_before", "repaid"]].mean().round(3)
print(past)
```

Women and men repay at the **same** rate, yet women were approved less often. Rural applicants were approved far less than their repay rate justifies. Now train a model to copy those decisions:

```
def fit_and_audit(target, features, label, weights=None):
    m = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))
    m.fit(train[features], train[target], logisticregression__sample_weight=weights)
    out = test.copy()
    out["approved"] = (m.predict_proba(out[features])[:, 1] >= 0.5).astype(int)
    auc = roc_auc_score(out["repaid"], m.predict_proba(out[features])[:, 1])     # judged on
real repayment
    area, gender = fairness_summary(group_report(out, "area", "approved")),
fairness_summary(group_report(out, "gender", "approved"))
    return {"model": label, "auc_vs_real_repayment": round(auc, 3),
            "area_eq_opp_gap": area["equal_opportunity_gap"], "area_disparate_impact":
area["disparate_impact"],
            "gender_eq_opp_gap": gender["equal_opportunity_gap"]}

train = train.assign(is_female=(train["gender"] == "female").astype(int), is_rural=
(train["area"] == "rural").astype(int))
test = test.assign(is_female=(test["gender"] == "female").astype(int), is_rural=(test["area"]
== "rural").astype(int))

results = [
    fit_and_audit("approved_before", FEATURES + ["is_female", "is_rural"], "copy officers, with
gender+area"),
    fit_and_audit("approved_before", FEATURES, "copy officers, gender+area removed"),
]
print(pd.DataFrame(results).to_string(index=False))
```

- **With** gender and area as inputs, the model copies both unfair patterns.

- **Removing** them fixes the gender gap — but **not** the area gap. Why?

Proxies

```
from sklearn.model_selection import cross_val_score
for target in ["is_rural", "is_female"]:
    auc = cross_val_score(LogisticRegression(max_iter=1000),
StandardScaler().fit_transform(train[FEATURES]),
                          train[target], cv=5, scoring="roc_auc").mean()
    print(f"can the other features guess {target}? AUC {auc:.2f}")
print(train.groupby("area")[["momo_tx_month", "salaried",
"income_rwf"]].mean().round({"momo_tx_month": 0, "salaried": 2, "income_rwf": -3}))
```

The other features predict **area** very well (mobile-money activity, salary and income all differ by area) but cannot predict **gender**. So the model learns "low mobile money → refuse", which is really "rural → refuse". Removing a sensitive column is **not enough** when proxies exist.

Three ways to reduce bias

```
# 1. Better labels: learn from real repayment, not from past decisions
results.append(fit_and_audit("repaid", FEATURES, "train on real repayment"))

# 2. Reweighing: give each (group, label) combination equal total weight
grp = train["area"] + "_" + train["approved_before"].astype(str)
weights = (1.0 / grp.map(grp.value_counts())).values * len(train) / grp.nunique()
results.append(fit_and_audit("approved_before", FEATURES, "copy officers + reweighing",
weights))

# 3. Remove the strongest proxy
no_momo = [f for f in FEATURES if f != "momo_tx_month"]
results.append(fit_and_audit("repaid", no_momo, "real repayment, no mobile-money feature"))

print(pd.DataFrame(results).to_string(index=False))
```

Read the table row by row:

- **Better labels** help the most. When real outcomes exist, learn from them — not from people's past judgements.
- **Reweighing** reduces the area gap of the "copy officers" model, at little cost.
- **Removing the strongest proxy** helps further here: disparate impact rises above 0.8 at almost no accuracy cost, because the model barely relied on mobile-money activity (Lesson 15 shows this). That is not always so — proxies often carry real, legitimate information, and removing them can cost accuracy. Measure before deciding.

A fourth method, **different thresholds per group**, can equalise approval rates, but it means treating people differently by group at decision time, which may be unlawful discrimination in many places. Get legal advice before using it.

Mitigation is a decision, not a button

There is no free lunch: each method moves accuracy and different fairness measures in different ways. Put the options in a table like the one above, discuss them with the decision-makers, choose, and **document why** (Week 4). Then keep monitoring fairness in production — it can drift too.

Key terms

- **Historical bias** — unfairness carried in from past decisions in the data.
- **Proxy** — a feature that stands in for a sensitive attribute.

- **Reweighing** — giving training examples weights to balance groups and labels.
- **Pre-, in-, post-processing** — fixing bias in the data, in training, or in the decisions.
- **Feedback loop** — model decisions shape the future data the model learns from.

Remember this: Bias enters through past decisions and proxies; removing sensitive columns is not enough — compare mitigations openly, prefer better labels, and document the choice.

Name: Date:

Activity

Task 1

Which single feature is the strongest proxy for area? Remove each feature in turn and measure.

Task 2

Change the reweighing to balance gender instead of area. What happens?

Task 3

Explain the feedback loop in lending in three sentences. How could the SACCO break it?

Task 4

Make a table of the five rows above for gender as well as area.

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. What is historical bias?
   - A. A history lesson
   - B. Unfairness carried in from past decisions recorded in the data
   - C. Old computers
   - D. Slow training
2. Why did removing the area column not remove the area gap?
   - A. The model ignored all features
   - B. Other features act as proxies for area
   - C. It was a bug
   - D. Area was never in the data
3. Why did removing gender fix the gender gap here?
   - A. Women were removed
   - B. No other feature could stand in for gender
   - C. Luck
   - D. The model used age
4. Which mitigation helped most in the lesson?
   - A. Training on real repayment instead of past decisions
   - B. Adding more features
   - C. Raising the threshold
   - D. Deleting rural applicants
5. What is reweighing?
   - A. Changing the threshold
   - B. Weighing applicants
   - C. Removing data
   - D. Giving training examples weights so groups and labels are balanced
6. Why be careful with different thresholds per group?
   - A. They always fail
   - B. They are slow
   - C. Treating people differently by group at decision time may be unlawful
   - D. They need a GPU

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.